

Reviewer Pack — Baker-Norine Rank Gap Bounds Tseitin Resolution Width

Entry 753c91c0c440 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 18:30:32 UTC

Baker-Norine Rank Gap Bounds Tseitin Resolution Width

Entry ID: 753c91c0c440 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 18:30:32 UTC

1. Conjecture

Field A (mathematical branch): Tropical Riemann-Roch: Baker-Norine divisor theory and chip-firing on graphs **Field B** (complexity object): Resolution width and DPLL tree size of Tseitin formulas on 3-regular expanders

Statement:

Let G be a connected 3-regular graph on n vertices with cyclomatic genus $g(G) = |E| - |V| + 1$, and let $c: V(G) \rightarrow \{0,1\}$ be a 0/1 vertex charge with $\sum_v c(v)$ odd, defining the unsatisfiable Tseitin CNF $\text{Tseitin}(G,c)$. View c as an effective divisor of degree $|c|$ on the combinatorial graph G and compute its Baker-Norine rank $r_{\text{BN}}(c)$ via Dhar's burning / q -reduction algorithm; let $D(G,c) := g(G) - r_{\text{BN}}(c)$ be the rank gap. Conjecture: for every such (G, c) , the fixed-order DPLL backtrack tree of $\text{Tseitin}(G,c)$ has at least $2^{\lfloor D(G,c)/3 \rfloor}$ leaves, and the maximum clause width touched in that DPLL run lies in the interval $[D(G,c)/3, 3D(G,c) + 5]$.

Rationale (proposer's reasoning):

Tseitin's parity obstruction is exactly the divisor-class obstruction $[c]$ in the Picard group of G under chip-firing equivalence, and Baker-Norine rank quantifies how 'movable' c is across the graph — the same combinatorial degree of freedom that resolution must traverse to derive the global parity contradiction. Tropical Riemann-Roch on graphs has been entirely absent from proof complexity, yet it provides a sharp non-spectral, non-zeta handle: the genus-minus-rank gap is graph-theoretically invariant under chip-firing moves and is conceptually orthogonal to Ihara-zeta/Laplacian eigenvalue invariants. If true, the conjecture explains existing BSW Tseitin-on-expander width lower bounds via the fact that for 0/1 divisors of degree $\sim n/2$ the Baker-Norine rank generically saturates near -1 , forcing $D(G,c) \sim n/2$.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 77e7fc666fbc392b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{8, 10, 12, 14\}$ with 30 seeds each (120 trials), for every trial compute $D = g(G) - r_{BN}(c)$, leaf-count L , and DPLL max width W . Require $\geq 80\%$ of trials per n satisfy $\log_2(L)/D \in [1/3, 3]$ AND $W/D \in [1/3, 3]$; no trial may exceed those ratios by factor 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formulas resolution width expander graphs chip-firing - Baker-Norine rank divisor graph DPLL Tseitin lower bound - chip-firing q-reduction Dhar burning proof complexity resolution

Top relevant hits considered: - [<http://arxiv.org/abs/0811.2800v2>] Parallel Chip-Firing on the Complete Graph: Devil's Staircase and Poincare Rotation Number - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/2407.15889v3>] Periods and atomic firing sequences of parallel chip-firing games on directed graphs - [<http://arxiv.org/abs/1806.18061v1>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/1906.08515v1>] An overview on the bipartite divisor graph for the set of irreducible character degrees - [<http://arxiv.org/abs/2206.09662v2>] On approximating the rank of graph divisors - [<http://arxiv.org/abs/2007.09707v1>] On lengths of burn-off chip-firing games - [<http://arxiv.org/abs/1507.03209v4>] On the complexity of the chip-firing reachability problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations, permutations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3_regular_graph(n):
        while True:
            edges = set()
            vertices = list(range(n))
            for v in vertices:
                neighbors = random.sample(vertices, 2)
                if (v, neighbors[0]) not in edges and (neighbors[0], v) not in edges:
                    edges.add((v, neighbors[0]))
```

```

        edges.add((v, neighbors[1]))
    G = {v: [] for v in vertices}
    for u, v in edges:
        G[u].append(v)
        G[v].append(u)
    if is_connected(G) and spectral_gap(G) >= 0.15:
        return G

def is_connected(G):
    visited = set()
    stack = [0]
    while stack:
        v = stack.pop()
        if v not in visited:
            visited.add(v)
            stack.extend(u for u in G[v] if u not in visited)
    return len(visited) == len(G)

def spectral_gap(G):
    n = len(G)
    A = [[0] * n for _ in range(n)]
    d = [sum(1 for _ in neighbors) for neighbors in G.values()]
    for v, neighbors in enumerate(G.values()):
        for u in neighbors:
            A[v][u] = 1 / math.sqrt(d[u] * d[v])

    eigenvalues_A = eigenvalues(A)
    return max(eigenvalues_A) - min(eigenvalue for eigenvalue in eigenvalues_A if eigenvalue > 0)

def eigenvalues(M):
    n = len(M)
    identity = [[Fraction(1, n) if i == j else Fraction(0) for j in range(n)] for i in range(n)]

    def matrix_multiply(A, B):
        result = [[sum(A[i][k] * B[k][j] for k in range(n)) for j in range(n)] for i in range(n)]
        return result

    def subtract_matrices(A, B):
        result = [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]
        return result

    def add_matrices(A, B):
        result = [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
        return result

    def scalar_multiply(M, c):
        result = [[c * M[i][j] for j in range(n)] for i in range(n)]
        return result

    def matrix_power(M, k):
        if k == 0:
            return identity
        elif k % 2 == 1:
            return matrix_multiply(M, matrix_power(M, k - 1))
        else:
            half_power = matrix_power(M, k // 2)

```

```

        return matrix_multiply(half_power, half_power)

def trace(M):
    return sum(M[i][i] for i in range(n))

def det(M):
    if n == 1:
        return M[0][0]
    elif n == 2:
        return M[0][0] * M[1][1] - M[0][1] * M[1][0]
    else:
        result = Fraction(0)
        for j in range(n):
            submatrix = [[M[i][k] for k in range(n) if k != j] for i in range(1, n)]
            result += (-1) ** j * M[0][j] * det(submatrix)
        return result

def gaussian_elimination(A):
    A_augmented = [row + [Fraction(1)] for row in A]
    n = len(A)
    for i in range(n):
        pivot_row = max(range(i, n), key=lambda r: abs(A[r][i]))
        A_augmented[i], A_augmented[pivot_row] = A_augmented[pivot_row], A_augmented[i]
        for j in range(i + 1, n):
            factor = A_augmented[j][i] / A_augmented[i][i]
            A_augmented[j] = [A_augmented[j][k] - factor * A_augmented[i][k] for k in range(n + 1)]
    return [row[:-1] for row in A_augmented]

eigenvalues = []
A_copy = [row[:] for row in M]
while len(A_copy) > 0:
    pivot_row = max(range(len(A_copy)), key=lambda r: abs(A_copy[r][r]))
    A_copy[pivot_row], A_copy[0] = A_copy[0], A_copy[pivot_row]
    pivot = A_copy[0][0]
    for j in range(1, len(A_copy[0])):
        A_copy[0][j] /= pivot
    for i in range(1, len(A_copy)):
        factor = A_copy[i][0]
        for j in range(len(A_copy[0])):
            A_copy[i][j] -= factor * A_copy[0][j]
    eigenvalues.append(pivot)
    A_copy = [row[1:] for row in A_copy[1:]]
return eigenvalues

def dhar_burning(G, c):
    n = len(G)
    q = random.choice(list(G.keys()))
    reduced_divisor = {v: 0 for v in G}
    queue = [(q, c[q])]
    while queue:
        u, val = queue.pop(0)
        if val > 0:
            reduced_divisor[u] += val
            for v in G[u]:
                reduced_divisor[v] -= val / len(G[u])
                if reduced_divisor[v] < 0:

```

```

        queue.append((v, -reduced_divisor[v]))
    return sum(reduced_divisor.values())

def tseitin_formula(G, c):
    n = len(G)
    literals = {v: f'x{v}' for v in G}
    clauses = []
    for v in G:
        if c[v] == 1:
            clauses.append([literals[v]])
        else:
            clauses.append([-literals[v]])
    for u, v in G:
        clauses.append([literals[u], -literals[v]])
        clauses.append([-literals[u], literals[v]])
    return clauses

def dpll(clauses):
    n = len(clauses)
    assignment = [None] * n
    stack = []

    def backtrack(level):
        if level == n:
            return True
        for literal in clauses[level]:
            var, sign = (literal[1:], -1) if literal.startswith('-') else (literal, 1)
            if assignment[var] is None:
                assignment[var] = sign
                stack.append((var, sign))
                if backtrack(level + 1):
                    return True
                assignment[var] = None
                stack.pop()
        return False

    return backtrack(0)

n_values = [8, 10, 12, 14]
results = []
for n in n_values:
    for _ in range(30):
        G = generate_3_regular_graph(n)
        c = {v: random.choice([0, 1]) for v in G}
        if sum(c.values()) % 2 == 0:
            continue
        r_BN = dhar_burning(G, c)
        D_G_c = n - r_BN
        Tseitin_clauses = tseitin_formula(G, c)
        leaf_count = 0
        max_width = 0

        def dpll_with_count(clauses):
            nonlocal leaf_count, max_width
            n = len(clauses)
            assignment = [None] * n

```

```

stack = []

def backtrack(level):
    if level == n:
        nonlocal leaf_count
        leaf_count += 1
        return True
    for literal in clauses[level]:
        var, sign = (literal[1:], -1) if literal.startswith('-') else (literal, 1)
        if assignment[var] is None:
            assignment[var] = sign
            stack.append((var, sign))
            max_width = max(max_width, len(literal))
            if backtrack(level + 1):
                return True
            assignment[var] = None
            stack.pop()
    return False

return backtrack(0)

dpll_with_count(Tseitin_clauses)
results.append({
    "metric_name": "D(G,c)",
    "metric_value": D_G_c,
    "instances_tested": 1,
    "conjecture_holds": True if (1/3 <= math.log2(leaf_count) / D_G_c <= 3 and 1/3 <=
    "counterexample": "" if (1/3 <= math.log2(leaf_count) / D_G_c <= 3 and 1/3 <= max_

})

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    return {
        "seed": seed,
        "metric_name": "D(G,c)",
        "mean_metric_value": mean_metric_value,
        "std_metric_value": std_metric_value,
        "support_fraction": support_fraction
    }
else:
    for result in results:
        if not result["conjecture_holds"]:
            return {
                "seed": seed,
                "metric_name": "D(G,c)",
                "mean_metric_value": mean_metric_value,
                "std_metric_value": std_metric_value,
                "support_fraction": support_fraction,
                "counterexample": result["counterexample"]
            }
    return {
        "seed": seed,
        "metric_name": "D(G,c)",

```

```

        "mean_metric_value": mean_metric_value,
        "std_metric_value": std_metric_value,
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2*i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["mean_metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["mean_metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.8) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(result["support_fraction"] < 0.8 for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["support_fraction"] < 0.8)
        counterexample = next(result["counterexample"] for result in results if not result["conjecture"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34a147e1.py", line 264, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34a147e1.py", line 182, in run_trial
    G = generate_3_regular_graph(n)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34a147e1.py", line 33, in generate_3_regular_graph
    if is_connected(G) and spectral_gap(G) >= 0.15:
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34a147e1.py", line 54, in spectral_gap
    eigenvalues_A = eigenvalues(A)
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34a147e1.py", line 119, in eigenvalues
    A_copy[0][j] /= pivot
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ZeroDivisionError in a custom eigenvalue routine during graph generation, before any trial data was produced. No support fraction could be computed, so the pre-registered criterion is unmet. | next: Replace the buggy hand-rolled eigenvalue computation with `numpy.linalg.eigvalsh` (or use `networkx` connectivity + a robust spectral gap) and rerun the 120-trial protocol.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	90732
2	propose	claude_max	opus	0	0	431381
3	preregistration	claude_max	opus	0	0	6478
4	novelty	claude_max	opus	0	0	3479
5	novelty	claude_max	opus	0	0	9544
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26195
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27043
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20407
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30731
10	judge	claude_max	opus	0	0	4623

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 650613 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/753c91c0c440.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/753c91c0c440.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/753c91c0c440.tar.gz` (if generated)